

Logística de Última Milla a Escala

Modifiability + Scalability + Cost — bounded contexts, OLTP/OLAP, ¿micro-servicios sí/no?

Ingeniería de Software II · ITBA · 2026-1C · Caso mock del wiki

RESUMEN

Scaleup de logística asset-light (USD 80M, 200K entregas/día, monolito Python que cruje). La pregunta del board — ¿microservicios sí o no? — es una trampa: la respuesta correcta es **bounded contexts primero, microservicios después**, separando decomposición lógica de decomposición física. Caso sintético (source: mock-logistica-ultima-milla.md).

§1 Enunciado

Startup de *logística de última milla* asset-light (sin flota propia; coordina **8.000 conductores independientes** vía app). 3 años, factura **USD 80M/año**, 6 ciudades del Cono Sur. Dos segmentos:

- **(a) E-commerce same-day** para grandes retailers: volumen alto, márgenes finos, API estable + SLAs estrictos.
- **(b) Entregas express** para restaurantes y comercios: volumen masivo, ventana corta, simplicidad y precio bajo.

Volumen: **200.000 entregas/día**, pico **35.000 entregas/hora** los viernes 19-22 hs. Cada entrega genera **~150 eventos** (creado, asignado, recogido, geoloc. cada 30 s, entregado, calificado) → **~30M eventos/día**. Equipo: de **25 a 90 ingenieros** en 18 meses. El monolito Python muestra grietas: deploys lentos, latencias intermitentes, squads pisándose. El CTO debate microservicios vs refactor interno; el board pide features; los inversionistas vigilan el costo de cloud (que crece más rápido que los ingresos). (source: mock-logistica-ultima-milla.md)

§2 Stakeholders

- Consumidores finales
- Vendedores / comercios (retailers grandes y pequeños)
- Conductores independientes (gig workers)
- Operaciones (incidentes en vivo)
- Pricing / matching
- Producto por segmento (retail vs restaurantes)
- Finanzas (cobranza B2B, settlement)
- Soporte al cliente
- Legal (regulación gig economy)
- Inversionistas (unit economics, costo cloud, churn)
- Carriers tradicionales (Andreani, OCA — eventual sub-contratación)

Stakeholders con intereses en tensión: retailers quieren estabilidad y SLAs; comercios quieren simplicidad y precio; conductores migran a competidores si el matching es injusto; inversionistas presionan el costo. Esto justifica priorizar atributos en conflicto (§3).

§3 Atributos de calidad priorizados

FUENTE

Tomados **exclusivamente** de la tabla ISO 25000 de la cátedra. Modifiability (Bass-Clements-Kazman) = Maintainability en la cátedra. Cost-efficiency y data-driven product son restricciones de negocio, no atributos. (source: mock-logistica-ultima-milla.md)

SCALABILITY

PERFORMANCE

AVAILABILITY

MAINTAINABILITY

INTEROPERABILITY

TABLA 1. Atributos priorizados y driver del enunciado

#	Atributo	Driver cuantificado
1	Scalability	Pico 5× sobre media; +100% anual sostenido.
2	Performance	Matching conductor-pedido < 5 s; geoloc. propagada < 10 s.
3	Availability	Un viernes 20 hs caído = millones + churn de conductores.
4	Maintainability	9 squads en paralelo sin bloquearse sobre el monolito.
5	Interoperability	APIs estables con docenas de retailers; carriers heterogéneos.

Secundarios del enunciado: Reliability (entrega perdida = dinero + redes sociales), Auditability (event log alimenta pricing y disputas), Supportability (diagnóstico en vivo). (source: mock-logistica-ultima-milla.md)

§4 Escenarios de calidad (cuantificados)

TABLA 2. Estímulo → respuesta → medida

Atributo	Escenario (estímulo + medida)
SCALABILITY	Viernes 19 hs la carga salta de 7K a 35K entregas/hora → el sistema absorbe el 5× sin degradar matching ni tracking, escalando por zona/horario (no por pico permanente).
PERFORMANCE	Un pedido entra en hora pico → se asigna conductor en < 5 s ; cada ping de geoloc. se propaga al cliente en < 10 s .
AVAILABILITY	Cae el nodo de tracking un viernes 20 hs → el pipeline de eventos (Kafka) bufferea y el CRUD operacional sigue tomando pedidos; sin pérdida de entregas en curso.
MAINTAINABILITY	9 squads despliegan en la misma semana → ningún deploy bloquea a otro contexto; cambios en Matching no rompen Facturación.

§5 Diagrama de componentes

Bounded contexts dentro del monolito modular; tracking en pipeline event-driven separado del CRUD; OLTP/OLAP desacoplados por ETL nocturno.

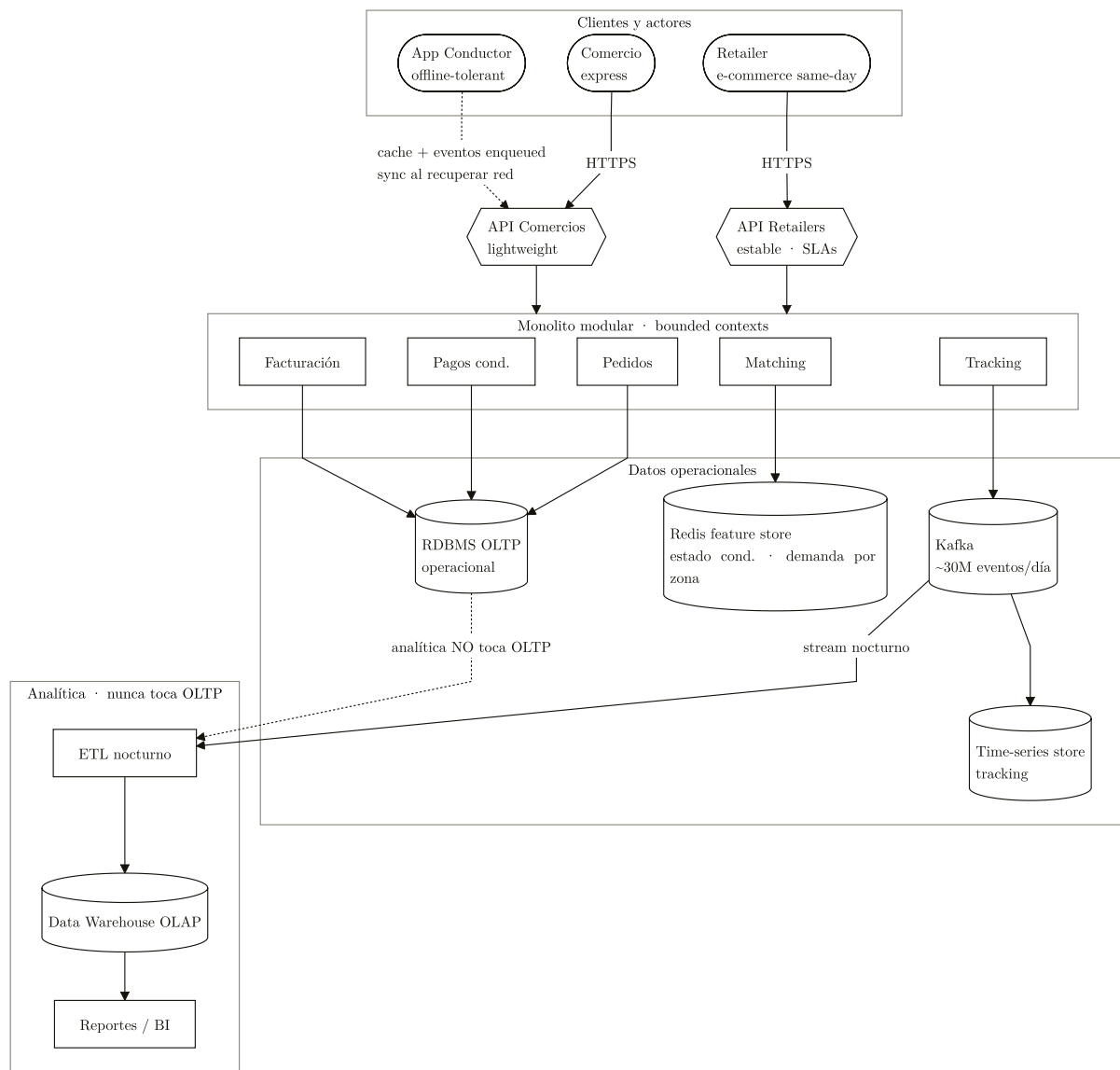


FIGURA 1. *Bounded contexts dentro del monolito modular; Tracking en pipeline event-driven (Kafka → time-series) separado del CRUD; OLTP y OLAP desacoplados por ETL nocturno; app de conductor offline-tolerant.*

§6 La pregunta clave: ¿microservicios sí o no?

DEFINICIÓN

Bounded context: frontera lógica donde un modelo de dominio es consistente y un equipo manda (DDD, Evans). **Microservicio:** frontera física — proceso/deploy/datos independientes. Confundirlas es el error central del caso.

TRADE-OFF

Maintainability ↔ overhead operativo: microservicios dan autonomía de deploy por squad, pero imponen red, observabilidad distribuida, consistencia eventual y DevOps que un equipo joven no tiene. Se resuelve obteniendo la autonomía con bounded contexts dentro del monolito y extrayendo a servicio solo el contexto que ya lo justifica por carga/contrato/fricción.

CUÁNDO NO • CUIDADO

Saltar a microservicios "porque el monolito cruje" es el anti-patrón #1: con bounded contexts mal definidos, microservicios = **deuda con extra-overhead**. "Microservicios sin bounded contexts es deuda con extra-overhead." (source: mock-logistica-ultima-milla.md)

Respuesta: No (todavía) como decisión global. Primero *decomposición lógica* (bounded contexts: Pedidos, Matching, Tracking, Pagos a conductores, Facturación). *Decomposición física* (extraer a microservicio) solo donde el perfil de carga ya diverge del CRUD: **Tracking** (30M eventos/día → pipeline propio) y **Matching** (latencia in-memory propia). El resto queda en el monolito modular hasta que los contextos sean estables y los equipos choquen. (source: mock-logistica-ultima-milla.md)

CUÁNDO SÍ • VENTAJAS

Cuándo extraer un bounded context a microservicio: contexto **estable** (la frontera dejó de moverse), **perfil de carga/latencia divergente** del resto (Tracking, Matching), equipos que **se bloquean** en deploys, o un **contrato de SLA** propio. No antes.

§7 Decisiones arquitectónicas + rationale

TABLA 3. Decisión → rationale (source: mock-logistica-ultima-milla.md)

Decisión	Rationale
Bounded contexts antes de microservicios	Microservicios prematuros con equipo chico = suicidio. Bounded contexts dentro del monolito dan autonomía sin overhead operativo; se extraen recién cuando son estables y los equipos chocan.
Tracking en pipeline event-driven (Kafka → time-series), separado del CRUD	30M eventos/día acoplados al RDBMS operacional destruyen performance. Patrón Partition/Replicate/Index ([[Partition, Replicate, Index]]).
Matching como servicio dedicado con feature store en Redis	Perfil de latencia/carga radicalmente distinto al CRUD; necesita motor in-memory propio (estado de conductores, demanda por zona).
API pública por segmento (Retailers estable / Comercios lightweight)	Mismo API para audiencias incompatibles genera fricción. Cada API evoluciona a su ritmo y SLA.
ETL nocturno stream → data warehouse; analítica nunca toca OLTP	Reportes consultan TBs históricos; el OLTP no sostiene queries analíticas. OLTP vs OLAP separados ([[OLAP y ETL]]).
Auto-scaling por zona horaria y día (viernes 19 hs ≠ martes 10 hs)	Cost-efficiency exige pagar capacidad real, no pico permanente.
App de conductor offline-tolerant	Pedidos cacheados + eventos enqueued y sincronizados al volver red: conductores entran a sótanos, túneles, zonas sin cobertura.

§8 Trampas y anti-patrones

CUÁNDO NO • CUIDADO

- Saltar a microservicios con 25 ingenieros (overhead > beneficio).
- Persistir 30M eventos de tracking/día en el RDBMS operacional.
- Mismo endpoint API para retailer enterprise y app de comercio chico.
- Reportes ejecutivos consultando producción (matan latencias de checkout).
- Cluster siempre escalado al pico (cost-efficiency destruida).
- App online-only para conductor (10% en zonas sin cobertura churra).
- Refactor "big bang" del monolito (deadline político, riesgo enorme).
- Optimizar pricing sin event log granular (decisiones a ciegas).
- Crecer de 25 a 90 ingenieros sin guardrails: reproduce el problema en lugar de resolverlo.

REGLA DE ORO

Regla: decomposición **lógica** (bounded contexts) es barata y reversible; decomposición **física** (microservicios) es cara y difícil de revertir. Hacer la lógica ya; la física, solo donde el dato (carga, latencia, SLA, choque de equipos) lo exija.

EN EL PARCIAL

Respuesta mínima aprobada: stakeholders + atributos priorizados con driver cuantificado + escenarios + diagrama con componentes nombrados + responder explícito

"¿microservicios sí/no?" distinguiendo **bounded context (lógico)** de **microservicio (físico)**. Error que mata la respuesta: recomendar microservicios global sin esa distinción, o dibujar cajas antes de articular atributos.

Pregunta a profundizar: ¿cuándo un bounded context "merece" extraerse? Criterio (volumen, equipo, frecuencia de deploy, contrato de SLA) y la trampa de la respuesta "depende" ([BDUF vs YAGNI vs JEDUF]). (source: mock-logística-última-milla.md)